

贪心算法

《算法设计与分析》

信息科学与技术学院 张德富



02



任务选择问题



二、任务选择问题

- 假定有一系列任务的集合 $S = \{a_1, a_2, \dots, a_n\}$ ，这些任务竞争同一资源，这个资源一次只能被一个任务占用。任务 a_i 占用该资源后，不能中断，直至完成。每个任务 a_i 有一个起始时间 s_i 和一个完成时间 f_i ，且 $0 \leq s_i < f_i < \infty$ 。如果任务被选择占用资源，则该任务可在半开时间区间 $[s_i, f_i)$ 内占用资源。如果任务 a_i 的活动区间 $[s_i, f_i)$ 和任务 a_j 的活动区间 $[s_j, f_j)$ 互不重叠，则称任务 a_i 和 a_j 是相互兼容的，即他们占用资源时，互不冲突。
- 任务选择(Task Select)问题就是在任务占用资源互不冲突的条件下，选择尽可能多的任务占用资源，即要选择相互兼容任务的最大子集。



二、任务选择问题

- 每一个任务需要使用同一资源

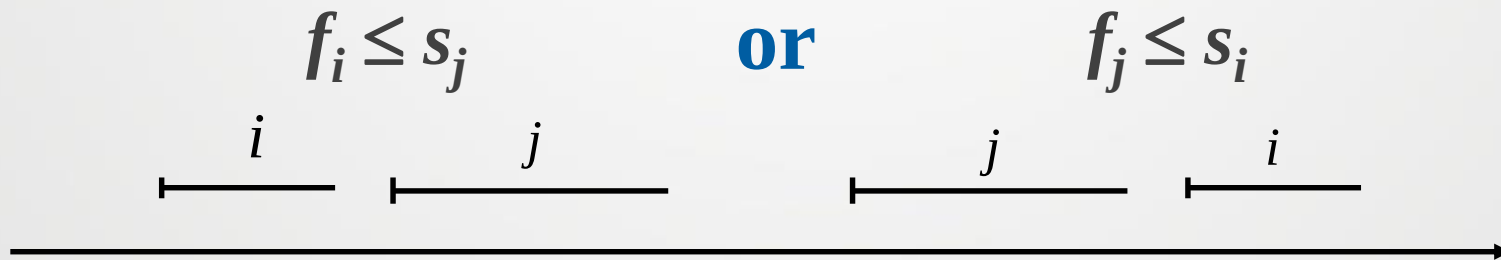
$S = \{a_1, \dots, a_n\}$ – 任务集合

- a_i 需要占用资源时间: $[s_i, f_i)$

- $s_i =$ 开始时间 and $f_i =$ 结束时间

- $0 \leq s_i < f_i < \infty$

- 任务 a_i 和 a_j 是可相容的如果时间段 $[s_i, f_i)$ 和 $[s_j, f_j)$ 没有重合





二、任务选择问题

- 选取最大的可相容的子集

- 例如：

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

- 任务已按照其结束时间升序排序
- 其中的一个可相容子集： $\{a_3, a_9, a_{11}\}$
- 最大可相容子集： $\{a_1, a_4, a_8, a_{11}\}$ and $\{a_2, a_4, a_9, a_{11}\}$

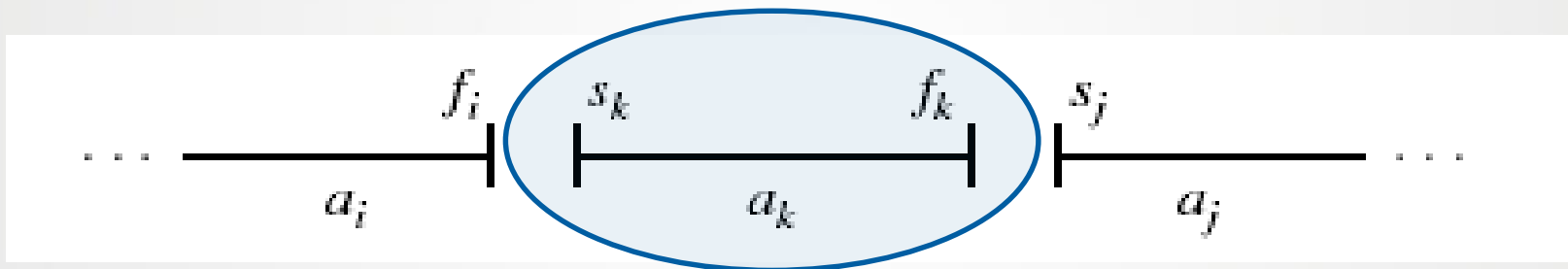


最优子结构

- 定义子问题区间:

$$S_{ij} = \{ a_k \in S : f_i \leq s_k < f_k \leq s_j \}$$

- 任务必须在 a_i 结束后开始并且在 a_j 开始前结束



- 任务和 S_{ij} 中的任务相容

- 所有的任务都在 f_i 前结束
- 所有的任务都 s_j 后开始



描述问题

- 增加假想任务

- $a_0 = [-\infty, 0)$

- $a_{n+1} = [\infty, \infty + 1)$

- S_{ij} 的范围 $0 \leq i, j \leq n + 1$

$S = S_{0,n+1}$ 整个的任务空间

- 我们假设在 S_{ij} 中任务已经按结束时间顺序升序排序:

$$f_0 \leq f_1 \leq f_2 \leq \dots \leq f_n < f_{n+1}$$

- 如果 $i \geq j$?

- 对于一个任务 $a_k \in S_{ij}: f_i \leq s_k < f_k \leq s_j < f_j$

与 $f_i \geq f_j$ 矛盾!

$\Rightarrow S_{ij} = \emptyset$ (必须为空!)

- 我们只需要考虑 $S_{ij}: 0 \leq i < j \leq n + 1$

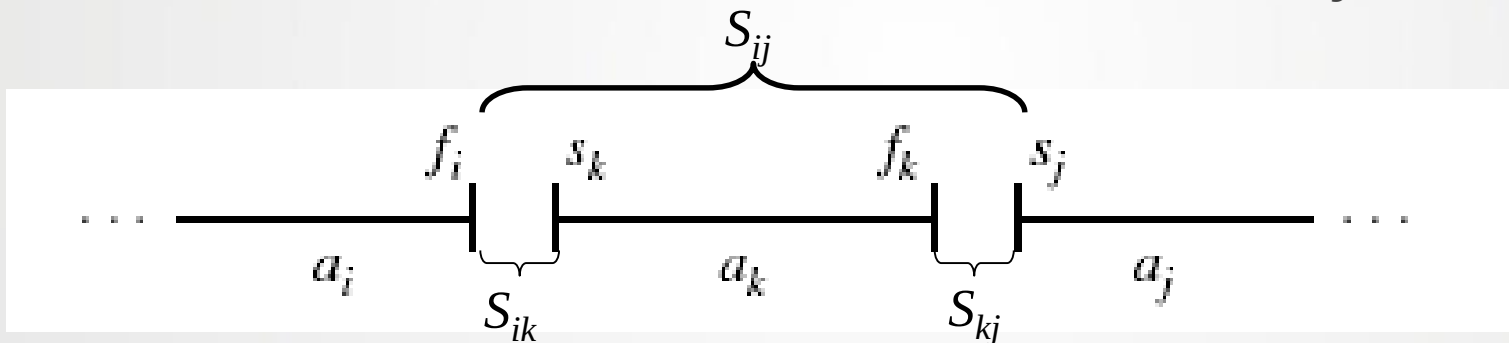


最优子问题

- 子问题:

- 从 S_{ij} 中选取一个最大可相容子集

- 假设对于上述子问题的一个解中包含 a_k (S_{ij} 非空)

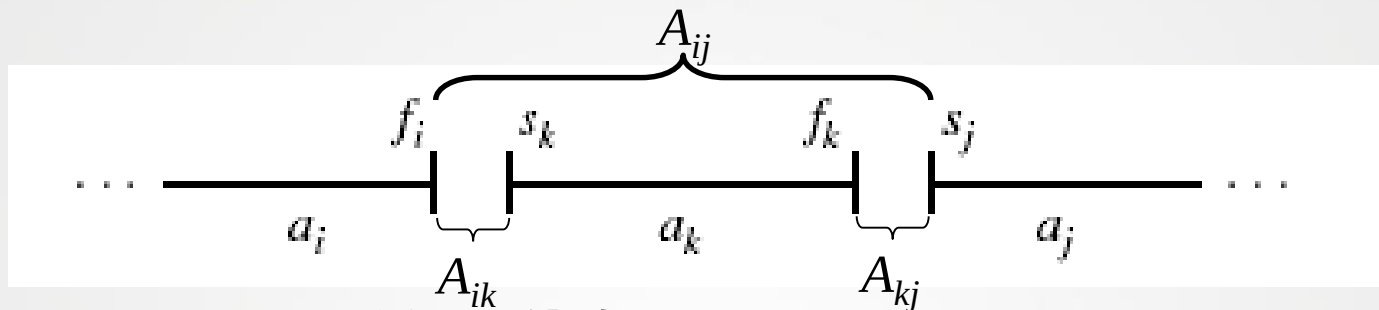


$$S_{ij} \text{ 的解} = (S_{ik} \text{ 的解}) \cup \{a_k\} \cup (S_{kj} \text{ 的解})$$

$$|S_{ij} \text{ 的解}| = |S_{ik} \text{ 的解}| + 1 + |S_{kj} \text{ 的解}|$$



最优子问题



A_{ij} 是 S_{ij} 的最优解

- **要求:** A_{ik} 和 A_{kj} 必须是最优解
- **假设存在** $\exists A_{ik}'$ **包含更多的任务**

$$\text{Size}[A_{ij}'] = \text{Size}[A_{ik}'] + 1 + \text{Size}[A_{kj}] > \text{Size}[A_{ij}]$$

⇒ **矛盾:** 我们曾经假设 A_{ij} 是 S_{ij} 的最优解

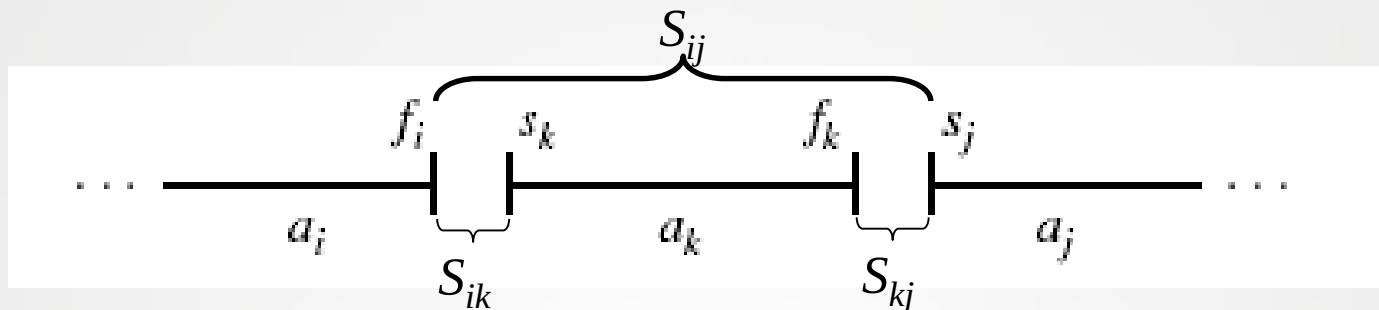


递归解法

- 任意问题的最优解包含它的子问题的最优解
- $c[i, j] = S_{ij}$ 的最大可相容子集的大小
- 如果 $S_{ij} = \emptyset \Rightarrow c[i, j] = 0 \ (i \geq j)$



递归解



如果 $S_{ij} \neq \emptyset$ 并且如果我们假设 a_k 在一个最优解中
(S_{ij} 的最大可相容子集)

$$c[i, j] = c[i, k] + c[k, j] + 1$$



递归解

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset \\ \max_{i < k < j, a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset \end{cases}$$

k 有 $j-i-1$ 种可能

- $k = i+1, \dots, j-1$
- a_k 不能是 a_i 或者 a_j (从 S_{ij} 的定义)

$$S_{ij} = \{ a_k \in S : f_i \leq s_k < f_k \leq s_j \}$$

- 我们检查各种可能的情况并选求最好的

我们可以写出一个动态规划算法



定理

定理7.2:

设 $S_{ij} \neq \emptyset$ 并且 a_m 是 S_{ij} 中最早完成的任务 : $f_m = \min \{ f_k : a_k \in S_{ij} \}$

有:

1. a_m 是的最大可相容子集中的一个 S_{ij} 的最大可相容子集中的一个
 - 存在一些包含 a_m 的最优解
2. $S_{im} = \emptyset$, 所以选择 a_m S_{mj} 是唯一的非空子问题



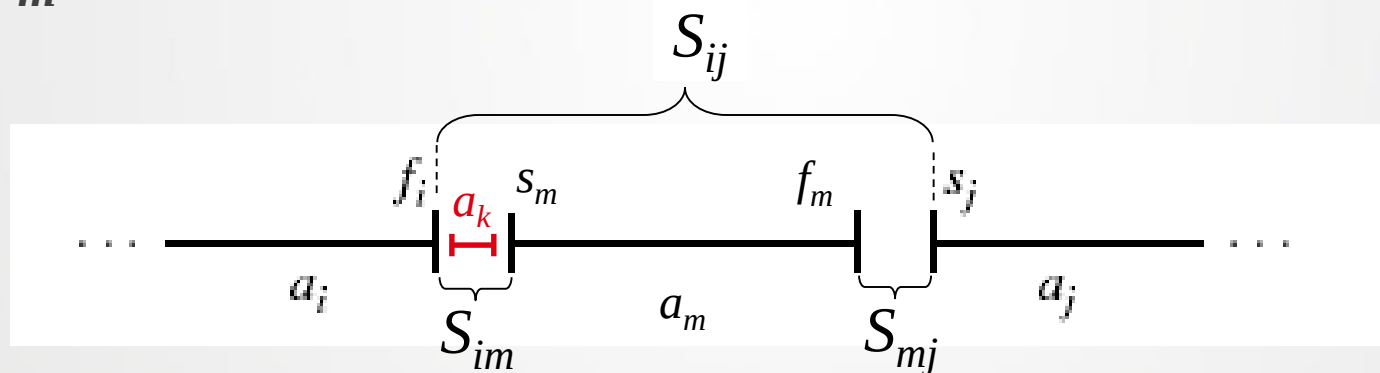
证明

2. 假设 $\exists a_k \in S_{im}$

$$f_i \leq s_k < f_k \leq s_m < f_m$$

$\Rightarrow f_k < f_m$ 矛盾!

a_m 并不是最早完成的

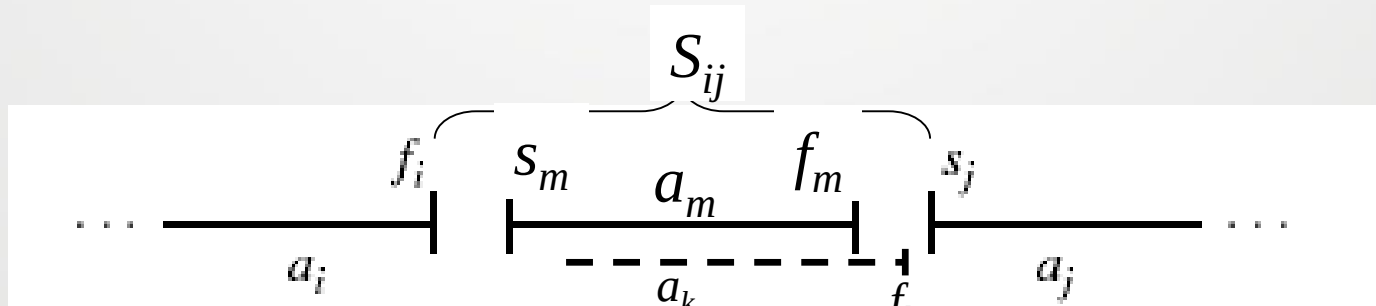


$\Rightarrow$ 没有 $a_k \in S_{im} \Rightarrow S_{im} = \emptyset$



Proof

1. S_{ij} 的最优解中的一个包含 a_m
 - $A_{ij} = S_{ij}$ 的最优解
 - 将任务按完成时间升序排序
 - 让 a_k 成为 $A_{ij} = \{a_k, \dots\}$ 的第一个值
 - 如果 $a_k = a_m$ 完成!
 - 否则 换成 a_m (建立一个新子集 A_{ij}')
 - 由于 $f_m \leq f_k$ A_{ij}' 中的任务继续可相容
 - A_{ij}' 和 A_{ij} 的大小一样 $\Rightarrow a_m$ 是在某些最大可相容子集里





这个定理为什么有用

	动态规划算法	用这个定理
最优解的子问题	2 个: S_{ik}, S_{kj}	1 个: S_{mj} $S_{im} = \emptyset$
需要考虑的 可选择的情况	$j - i - 1$ 种	1 种: S_{ij} 中最先完成的

- 做贪心选择 (最先完成时间的选择)
 - 减少子问题和选择的数量
 - 由自顶向下的顺序解决每个子问题

谢谢大家